NetSage AI: Automated Cisco Packet Tracer Diagnostic & Troubleshooting Framework

Project Title: NetSage AI: Automated Cisco Packet Tracer Diagnostic & Troubleshooting Framework
Student Names: Atharva Jadhav, Anshul Bhusal, Ahad Gour
Institution: MIT Academy of Engineering (MITAOE)
Domain: Computer Networking & Artificial Intelligence / Responsible AI Audit
Topology: Dual-Subnet Router & Switch Campus Network
Target Cases: 20 Network Fault Troubleshooting Cases (OSI Layers 1 to 7)

---

1. Project Overview & Objectives

What is NetSage AI?

NetSage AI is an intelligent, automated network diagnostic system. It helps network engineers and students quickly find, understand, fix, and verify network configuration problems in Cisco Packet Tracer topologies.

Why is this Project Important?

In modern computer networks, troubleshooting network failures manually can take a lot of time and lead to human errors. NetSage AI solves this problem by combining three powerful techniques:

1. **Automated AI Diagnosis:** Uses an AI assistant to analyze command outputs and suggest exact CLI fix steps.
2. **Python Rule Checker:** Uses automated Python code to catch common network errors like duplicate IPs, wrong subnet masks, down interfaces, and missing routes.
3. **Human Review & Responsible AI Audit:** Checks all AI answers with human experts to ensure the AI does not make mistakes or give incorrect advice.

---

2. Network Topology & Baseline Configuration

Network Layout

The baseline network topology is an enterprise campus network connecting two local subnets (`LAN 1` and `LAN 2`) through a Cisco Router (`Router0`) and two Cisco Switches (`Switch0` and `Switch1`).

```
[ PC0 ] (192.168.1.10/24)
   |
[ Switch0 ] (Fa0/1 Access, Fa0/24 Uplink)
   |
[ Router0 ] --- Gig0/0/0: 192.168.1.1 (Gateway for LAN 1)
   |        --- Gig0/0/1: 192.168.2.1 (Gateway for LAN 2)
   |
[ Switch1 ] (Fa0/1 Access, Fa0/24 Uplink)
   |
[ PC1 ] (192.168.2.10/24)
```

Baseline Configuration Table

Device Name	Interface	IP Address	Subnet Mask	Default Gateway	DNS Server	Connected Device
PC0	FastEthernet0	`192.168.1.10`	`255.255.255.0`	`192.168.1.1`	`192.168.1.2`	Switch0 (Fa0/1)
PC1	FastEthernet0	`192.168.2.10`	`255.255.255.0`	`192.168.2.1`	`192.168.1.2`	Switch1 (Fa0/1)
Router0	Gig0/0/0	`192.168.1.1`	`255.255.255.0`	N/A	N/A	Switch0 (Fa0/24)
Router0	Gig0/0/1	`192.168.2.1`	`255.255.255.0`	N/A	N/A	Switch1 (Fa0/24)
Switch0	Fa0/23	N/A	N/A	N/A	N/A	Trunk Link to Switch1

In the baseline working state, **PC0** can successfully ping **PC1** with 0% packet loss.

---

3. Key Components of the Project

The project is built on 6 main modules:

1. **Master Cases Dataset (`cases.csv`):**

   Contains 20 structured networking failure cases spanning OSI Layers 1 to 7, plus 1 baseline operational case.

2. **Live Executed Packet Tracer Evidence (`evidence/`):**
   12 active failure scenarios were injected and tested live in Cisco Packet Tracer. Screenshots were captured proving:
   - Initial ping failure (broken state)
   - Diagnostic command output
   - Successful ping verification (fixed state)
3. **AI Diagnostic Prompt Engine (`diagnose_prompt.md`):**
   A structured system prompt that forces the AI assistant to give clean JSON output containing:
   - `root_cause` : What exact issue is breaking the network
   - `confidence` : High, Medium, or Low
   - `evidence` : Direct facts quoted from CLI output
   - `next_command` : Next command to run if confidence is limited
   - `fix_steps` : Step-by-step Cisco CLI fix commands
4. **Python Rule Checker (`rule_checker.py`):**
   A standalone Python program that checks device configurations against 6 fundamental networking rules to automatically detect errors.
5. **Responsible AI & Human Review Log (`responsible_ai_log.csv`):**
   Tracks human audit reviews of every AI answer, marking each case as **Accepted**, **Edited**, or **Rejected**.
6. **Analytics Dashboard (`generate_dashboard.py` & `dashboard/`):**
   Generates interactive HTML charts, Excel workbooks, and metric summaries showing project accuracy and performance.

---

4. Python Rule Checker Implementation (`rule_checker.py`)

The rule checker is written in Python (`rule_checker.py`). It scans device state dictionaries and CLI outputs to detect network faults automatically.

Rules Implemented in Python:

1. **Duplicate IP Address Check:**
   Scans all router interfaces and host IP configurations to find if the same IP is assigned to multiple devices.

2. **Subnet Mask Mismatch Check:**
   Checks if hosts on the same VLAN/subnet have conflicting subnet mask values.
3. **Default Gateway Mismatch Check:**
   Compares host default gateway settings with the actual IP address configured on the router interface.
4. **Interface Down Check:**
   Detects interfaces that have an assigned IP address but are marked as `"administratively down"`.
5. **Missing VLAN Check:**
   Checks if switch access ports are assigned to a VLAN that does not exist in the switch VLAN database.
6. **Missing Route Check:**
   Verifies if gateway routers have static or dynamic routes to target destination subnets.

---

5. Overview of the 20 Network Fault Cases

The project tests 20 distinct network fault cases across all OSI layers:

Case ID	Fault Title	OSI Layer	Concept	Status
CASE-000	Baseline Working State Verification	Layer 3	Operational Topology	Operational
CASE-001	Router Interface Shutdown (Gig0/0/0)	Layer 1	Interface Status	Live Executed
CASE-002	Default Gateway Mismatch on PC0	Layer 3	Default Gateway	Live Executed
CASE-003	Subnet Mask Mismatch on PC1	Layer 3	Subnetting	Live Executed
CASE-004	Duplicate IP Conflict on PC1	Layer 3	IP Addressing	Live Executed
CASE-005	Access Port Shutdown on Switch0 (Fa0/1)	Layer 2	Interface Status	Live Executed
CASE-006	VLAN Missing from Trunk Allowed List	Layer 2	802.1Q Trunking	Live Executed
CASE-007	Uplink Port Shutdown on Switch0 (Fa0/24)	Layer 2	Interface Status	Live Executed
CASE-008	Wrong Host IP Subnet on PC1	Layer 3	IP Addressing	Live Executed

---

Case ID	Fault Title	OSI Layer	Concept	Status
CASE-009	Missing Default Gateway on PC1	Layer 3	Default Gateway	Live Executed
CASE-010	Router Outbound ACL ICMP Block	Layer 4	Access Control List	Live Executed
CASE-011	DHCP Pool Default Gateway Misconfiguration	Layer 7	DHCP Services	Live Executed
CASE-012	Inter-Switch Trunk Port Shutdown on Switch1	Layer 2	Interface Status	Live Executed
CASE-013	Outbound Extended ACL ICMP Deny Rule	Layer 4	ACL Filtering	Reference Case
CASE-014	NAT Overload Access-List Subnet Exclusion	Layer 3	NAT / PAT	Reference Case
CASE-015	Unassigned Gateway in DHCP Pool	Layer 7	DHCP Services	Reference Case
CASE-016	Port Security Violation Err-Disabled State	Layer 2	Port Security	Reference Case
CASE-017	Duplex Mismatch Late Collisions	Layer 1	Physical Parameters	Reference Case
CASE-018	Client DNS Server IP Misconfiguration	Layer 7	DNS Services	Reference Case
CASE-019	OSPF Duplicate Router-ID Conflict	Layer 3	Dynamic Routing	Reference Case
CASE-020	Router Subinterface Dot1Q VLAN Mismatch	Layer 2	Router-on-a-Stick	Reference Case

---

6. Human Review & Responsible AI Audit Results

Every AI diagnosis generated during the project was evaluated by human network reviewers in `responsible_ai_log.csv`.

Review Decision Summary:

- **Accepted (16 Cases - 76.2%):**
  The AI correctly identified the root cause and gave exact working CLI fix commands (e.g., `CASE-001` interface shutdown, `CASE-002` gateway mismatch, `CASE-010` ACL block).

- **Edited (3 Cases - 14.3%):**
  The AI diagnosed the problem correctly, but the human reviewer added missing CLI cleanup or recovery steps:
  - *CASE-004:* Added switch MAC address table check (`show mac address-table`) to verify physical host attachment.
  - *CASE-014:* Added cleanup command (`no access-list 1`) before re-adding the correct NAT ACL rule.
  - *CASE-016:* Added mandatory interface reset commands (`shutdown` followed by `no shutdown`) required to recover an err-disabled port.
- **Rejected (2 Cases - 9.5%):**
  The AI gave an incorrect diagnosis due to misreading CLI evidence:
  - *CASE-019 (OSPF Router ID Collision):* AI incorrectly diagnosed physical cable flapping. Human reviewer corrected it to **OSPF Router ID conflict** (`1.1.1.1` on both routers) based on the syslog error `%OSPF-4-DUP_RTRID` .
  - *CASE-020 (Subinterface Encapsulation Mismatch):* AI misread subinterface number `.20` as VLAN ID 10 and suggested renaming the subinterface. Human reviewer corrected it to change `encapsulation dot1Q 20` .

---

7. Key Deliverables & Code Files

All required project files are saved in the main project directory:

File Name	Description
`report.md`	Complete official project report (this document).
`packet_tracer/NETSAGEPACKETTRACER(BASE).pkt` , `packet_tracer/NETSAGEPACKETTRACER(AFTERCASES).pkt`	Cisco Packet Tracer topology network file.
`rule_checker.py`	Python script for automated deterministic rule checking.
`generate_dashboard.py`	Python script for generating HTML dashboard & analytics.
`diagnose_prompt.md`	AI diagnostic system prompt with JSON response schema.
`cases.csv`	Master dataset containing all 20 network troubleshooting cases.
`responsible_ai_log.csv`	Responsible AI human audit log tracking accuracy and edits.

File Name	Description
`dashboard/dashboard.html`	Interactive visual HTML dashboard report.

---

8. How to Run and Verify the Project

Running the Python Rule Checker:

Open terminal/PowerShell in the project folder and run:

```
python rule_checker.py
```

Output: Scans sample network topology and prints detected faults along with recommended CLI fixes.

Running the Dashboard Generator:

Run the following command to update HTML dashboard charts and metrics:

```
python generate_dashboard.py
```

Output: Creates `dashboard/dashboard.html`, `dashboard/dashboard.xlsx`, and `dashboard/dashboard_chart.png`.

---

9. Conclusion

The **NetSage AI Framework** successfully demonstrates how artificial intelligence and Python automation can streamline network troubleshooting in Cisco enterprise environments. By combining automated Python rule checking, structured AI diagnostics, and human-in-the-loop auditing, the system achieves high diagnostic accuracy while ensuring safety, transparency, and operational reliability.